

Slide 1:-

Our technical knowledge is stored in XML documents. These documents are connected using doc: and doc:wiki: references. A normal keyword search can find matching documents, but it cannot understand the connections between them. Also, an LLM may give incorrect answers if it doesn't receive the right context. So, our goal is to build a Knowledge Graph-based GraphRAG system that retrieves connected documents and provides accurate technical answers.

Slide 2:-

Our solution is to convert the XML documents into a Knowledge Graph. Each XML document becomes a node, where the doc_id is used as the unique identifier and the document title is used as the node name. We then parse the XML content to extract references such as doc, doc:wiki, and doc:Temp_Import. These references define the relationships between XML documents and become the edges in the graph. This allows us to represent the complete document network instead of treating each XML document independently

Slide 3:-

Slide 3 – End-to-End Pipeline

Phase 1 – Data Preparation (Steps 1–4)

"The first phase is data preparation."

Step 1 – Install Libraries

- Install all required Python libraries.
- Import the required packages.

↓

Step 2 – Read Evaluation Dataset

- Read the rag_eval_set table.
- This contains the user questions and XML source paths.

↓

Step 3 – Match XML Documents

- Match the XML source paths with the XML files stored in knowbot_raw_data.
- Load the required XML documents.

↓

Step 4 – Clean XML

- Remove unwanted spaces.
 - Remove invalid characters.
 - Clean the XML content for processing.
-

Phase 2 – Knowledge Graph Construction (Steps 5–8)

"The second phase converts XML documents into a Knowledge Graph."

Step 5 – XML Reference Parser

- Parse the XML content.
- Identify document references.
- Detect doc, doc:wiki, and doc:Temp_Import links.

↓

Step 6 – Extract References

- Extract every document reference.
- Identify which XML document points to another XML document.

↓

Step 7 – Convert to RDF Triples

Convert every relationship into:

Subject → Predicate → Object

Example

Engine

doc:wiki

↓

Fuel System

↓

Step 8 – Build Knowledge Graph

- Each XML document becomes a node.
 - Extracted references become edges.
 - Build the complete Knowledge Graph.
-

Phase 3 – Graph Optimization (Steps 9–10)

"The third phase improves and validates the graph."

Step 9 – PageRank

- Calculate the importance score of each document.
- Frequently connected documents receive higher PageRank.



Step 10 – Visualization

- Display the Knowledge Graph.
 - Validate that all document relationships are correct.
-

Phase 4 – Graph Retrieval (Steps 11–12)

"The fourth phase retrieves the most relevant knowledge."

Step 11A – Candidate Retrieval

- Process the user query.
- Compare the query with all indexed documents.
- Select the Top-K candidate documents.



Step 11B – Graph Traversal

- Start from the candidate nodes.
- Traverse connected documents using BFS.
- Retrieve related nodes within two hops.



Step 11C – Context Builder

- Collect the retrieved documents.
- Build a structured context for the LLM.



Step 12 – GraphRAG Pipeline

- Combine Candidate Retrieval.

- Graph Traversal.
 - Context Builder.
 - Return the retrieved subgraph and context.
-

Phase 5 – Answer Generation (Steps 14–15)

"The final phase generates the answer."

Step 14 – LLM

- Send the retrieved context to Azure OpenAI.
- The LLM reads only the retrieved knowledge.

↓

Step 15 – Generate Answer

- Generate the final technical answer.
- Display the retrieved subgraph for explainability.

slide 4

"After cleaning the XML data, the next step is to identify how XML documents are connected. Every XML document contains content, and inside that content there are references such as doc, doc:wiki, and doc:Temp_Import. The parser scans the entire content, extracts all these references, and normalizes them by removing anchors, sections, and extra spaces. Finally, for each XML document, we store the relationship type and the referenced document. This output becomes the input for RDF Triple generation in the next stage."

Slide 5

In the previous step, we extracted all the document references from the XML content, such as doc, doc:wiki, and doc:Temp_Import. However, these references are still text values, so they cannot be directly used to build a Knowledge Graph.

The purpose of this step is to convert those extracted references into RDF triples, which is a standard way to represent relationships in a Knowledge Graph.

In our project, every XML document has a unique **Doc_ID**. We first resolve each extracted reference to its corresponding Doc_ID using the reference resolution dictionary created earlier.

After resolving the reference, we create an RDF triple with three components:

- **Subject** – the parent XML document (Parent Doc_ID)
- **Predicate** – the relationship type, such as doc, doc:wiki, or doc:Temp_Import
- **Object** – the referenced XML document (Child Doc_ID)

For example, if Document 101 references Document 205 using doc:wiki, the RDF triple becomes:

(101, doc:wiki, 205).

We also remove invalid references and self-loops to ensure the graph is clean.

Finally, these RDF triples become the relationships, or edges, when we build the Knowledge Graph in the next step."

Slide 6:-

After generating the RDF triples, the next step is to build the Knowledge Graph. In our project, we use the NetworkX library and create a Directed Graph because document references have a direction—from one XML document to another. Every XML document becomes a node, where the Doc_ID is used as the unique identifier. Along with the node, we store additional metadata such as the title, reference, version, author, and content. Next, each RDF triple is converted into an edge connecting two document nodes. The relationship type, such as doc, doc:wiki, or doc:Temp_Import, is stored as an edge attribute. Finally, we validate the graph by checking for invalid references and self-loops, and we calculate graph statistics such as the total number of nodes, edges, connected nodes, and graph density. The completed Knowledge Graph becomes the foundation for GraphRAG retrieval.

Slide 7:- diagram image

Slide 8:-

The retrieval process starts when the user submits a technical question. First, the query is cleaned by removing stop words and extracting the important keywords. These keywords are compared with every XML document in the graph index. Each document receives a retrieval score based on keyword overlap, PageRank, graph degree, title matching, and reference matching. The highest-ranked documents become the starting points for graph traversal. From these candidate documents, we perform a Breadth-First Search over the Knowledge Graph, exploring both outgoing and incoming relationships up to two hops. This

allows us to retrieve not only the matching document but also the related documents connected through the Knowledge Graph

Slide 9:-

After graph traversal, we obtain a set of connected documents. The next step is to prepare the information for the LLM. The retrieved documents are first sorted by their retrieval score, duplicate documents are removed, and the XML content is extracted from each node. To keep the prompt efficient, only the highest-ranked documents are selected. Finally, all the retrieved content and document metadata are combined into a single structured context, which is then passed to the language model for answer generation.

Slide 10 diagram

Slide 11

After implementing the individual retrieval components, the next step is to combine them into a single GraphRAG retrieval pipeline. The GraphRAG class acts as the orchestrator of the retrieval process. When a user submits a query, it first calls Candidate Retrieval to identify the most relevant documents. It then performs Knowledge Graph Traversal to discover related documents through graph relationships. Next, the Context Builder prepares a structured context by combining the retrieved document content and metadata. Finally, the pipeline creates a retrieved subgraph and packages the query, retrieved nodes, subgraph, and graph context into a single retrieval result. This retrieval package is then passed to the LLM for answer generation

Slide 12 :-

This slide shows the execution of the complete GraphRAG retrieval pipeline. After the user submits a technical question, the GraphRAG object is initialized with the Knowledge Graph and graph index. The retrieval pipeline is then executed, which internally performs Candidate Retrieval, Graph Traversal, and Context Building. The output consists of the retrieved documents, the relationships between those documents, the retrieved subgraph, and the retrieval statistics. This information is then prepared for the LLM

Slide 13:

After retrieval is complete, the Graph Context is converted into a prompt for the LLM. The prompt includes the user question, the retrieved documents, the relationships between those documents, and a system instruction that tells the LLM to answer only from the retrieved Knowledge Graph. This constraint helps reduce hallucinations because the model is limited to the retrieved context. The prompt is sent to Azure OpenAI, which generates the final technical answer. We also visualize the retrieved subgraph so users can understand which documents and relationships were used to produce the answer.